Retail CCTV Store Mapping & Customer Heatmap

A computer-vision system for reconstructing a camera-visible retail layout, tracking customer movement, and generating a time-synchronized 2D heatmap.

Input	2–3 minute fixed CCTV video clip
Primary Output	Approximate 2D store map + animated customer heatmap
Core CV Tasks	Scene understanding, shelf/layout extraction, person detection, multi-object tracking, perspective mapping
Key Constraint	The generated map represents only the camera-visible area and is explicitly approximate.
Recommended MVP	Manual floor calibration first; automate layout reconstruction in later phases.

Project principle: do not attempt to hallucinate a complete floor plan. The system should reconstruct and label the portion of the store that can be supported by the CCTV view.

Proposal • September 2026

1. Executive Summary

This project proposes an end-to-end computer-vision pipeline that accepts a short CCTV video from a retail store and converts it into an approximate, camera-visible 2D representation of the store. The system identifies structural elements such as shelves/racks and walkable areas, detects and tracks customers, projects their movement onto the 2D representation, and generates an animated heatmap showing where customers spent time or passed through during the video.

The system is deliberately designed around uncertainty. A single CCTV camera may not see the complete store, may contain occlusions, and may provide strong perspective distortion. Therefore, the output is not presented as an exact architectural floor plan. Instead, it is a camera-visible store map with an explicit boundary for observed versus unknown areas.

The proposed implementation separates computer vision from map rendering and business analytics. This creates a modular architecture that can first be delivered as a reliable MVP and then progressively automated.

2. Objectives

- Accept a 2–3 minute CCTV video clip as the primary input.
- Extract and understand visible store structure, especially shelves/racks, walls, obstacles, and walkable regions.
- Create a rough 2D top-down representation of the camera-visible area.
- Detect customers and assign persistent tracking IDs across frames.
- Convert image-space customer trajectories into map-space trajectories.
- Generate a spatial heatmap based on customer presence and dwell/movement density.
- Provide heatmap playback synchronized with the original video timeline.
- Expose machine-readable trajectories and layout data so the frontend can render or analyze them.
- Clearly represent uncertainty and unseen areas instead of fabricating store geometry.

3. Scope and Assumptions

Initial scope: one fixed CCTV camera, one short video clip, indoor retail environment, ordinary customer traffic, and a camera with enough visible floor area to estimate movement. The first MVP does not attempt multi-camera stitching, full 3D reconstruction, product-level recognition, facial recognition, or exact architectural measurement.

Important assumption: the camera remains fixed during the clip. If the camera moves, the pipeline needs an additional camera-motion estimation/stabilization stage.

4. Proposed System Architecture

The recommended design uses Python for computer vision and video processing, with a Spring Boot service responsible for APIs, job orchestration, metadata, and integration with the user interface. Angular or React can provide the interactive visualization layer.

High-level flow
CCTV MP4 → Frame Sampling → Scene/Structure Analysis → Rough 2D Map → Person Detection → Multi-Object Tracking → Perspective/Map Projection → Trajectory Aggregation → Heatmap Engine → Interactive UI + Heatmap Video

Core modules

- Video Ingestion: validates video, extracts metadata, samples frames, and prepares processing jobs.
- Scene Analysis: identifies shelves/racks and other visible structural elements.
- Map Builder: converts scene geometry into a normalized 2D map representation.
- Person Detection: detects people in frames using a modern object-detection model.
- Tracking: maintains a stable identity for each person across frames.
- Projection: transforms image coordinates to floor/map coordinates using calibration or homography.
- Heatmap Engine: aggregates occupancy/trajectory points and produces time-dependent density maps.
- Visualization: overlays heatmap, customer paths, map structure, and playback controls.

5. Computer-Vision Pipeline

The pipeline should be implemented incrementally. Each stage should produce inspectable artifacts so failures can be diagnosed without rerunning the entire system.

Stage A — Video preprocessing

- Read the source video using OpenCV/FFmpeg.
- Record resolution, frame rate, duration, and keyframe information.
- Sample frames at approximately 5–10 FPS for analysis rather than running expensive inference on every source frame.
- Keep the original video for final synchronized rendering.

Stage B — Store structure extraction

The system detects visible shelves/racks and other structural features. Object detection can provide candidate regions, while segmentation or geometric post-processing can refine their boundaries. The output is converted from raw model predictions into structured geometry rather than directly asking a generative model to draw a floor plan.

Example internal representation:
StoreMap → boundary, shelves[], walls[], obstacles[], walkable_area, confidence, unseen_regions

Stage C — Approximate 2D map generation

For the MVP, the most reliable route is to define a floor-plane mapping using a small number of manually selected reference points. A homography/perspective transform then maps image coordinates to a normalized top-down plane. This removes much of the perspective distortion present in CCTV footage.

After the MVP works, automatic estimation of the floor plane and visible store geometry can be introduced. This staged approach reduces the risk of building an unreliable end-to-end system before the tracking and heatmap components are validated.

Stage D — Person detection and tracking

A modern person detector can identify customers in each sampled frame. A multi-object tracker such as ByteTrack or BoT-SORT can then maintain IDs and trajectories. The system should store at least the center/bottom-center of each person bounding box because the point closest to the floor is a more useful approximation of their location on the floor plane than the bounding-box center.

Stage E — Trajectory projection

For each tracked customer position, the system applies the floor-plane transform to obtain a normalized 2D map coordinate. The resulting data can be represented as a time series: customer ID, timestamp, x, y, confidence.

Stage F — Heatmap generation

The heatmap is created by accumulating projected customer positions over time. A density kernel/Gaussian blur or kernel-density-estimation approach can smooth the raw points. The implementation should support both a cumulative heatmap and a sliding-window heatmap so users can inspect how traffic changes over time.

6. Data Model and Intermediate Artifacts

A structured intermediate representation makes the project easier to debug, extend, and connect to a web application.

Artifact	Purpose	Example
video_metadata.json	Video properties and processing status	fps, duration, resolution
store_map.json	Camera-visible layout geometry	boundary, shelves, walkable area
tracks.json	Customer trajectories	ID, timestamp, x, y, confidence
heatmap_frames/	Time-indexed heatmap images	frame_0001.png ...
heatmap.mp4	Playback-ready visualization	timeline synchronized video

7. Interactive Output

The web interface should provide an interactive view rather than only a final static image.

- 2D Store Map: visible boundary, shelves/racks, walkable regions, camera position, and confidence/unseen boundary.
- Customer Overlay: optional live playback of tracked customer paths.
- Heatmap: cumulative and time-window modes.
- Timeline: a slider that lets the user inspect the heatmap at any point in the source video.
- Analytics panel: total tracked customers, estimated dwell time, high-traffic areas, and movement statistics.
- Export: PNG/SVG map, JSON trajectory data, and MP4 heatmap video.

8. Recommended Technology Stack

Layer	Technology	Reason
Video/CV	Python, OpenCV, FFmpeg	Strong ecosystem for video and image processing
Detection	YOLO-family detector or equivalent	Fast person/structure detection
Tracking	ByteTrack / BoT-SORT	Multi-object identity tracking
Geometry	NumPy, OpenCV, SciPy	Homography, transformations, density calculations
Backend	Spring Boot	Job APIs, orchestration, metadata, integration
Frontend	Angular or React	Interactive map, timeline, playback
Storage	Local filesystem initially; object storage later	Video and generated artifacts

Layer	Technology	Reason
Deployment	Docker; GPU-enabled host when needed	Repeatable deployment and acceleration

9. Development Roadmap

The project should be built as a sequence of increasingly automated milestones.

Phase 1 — Video + Tracking MVP

Ingest video, sample frames, detect people, track IDs, and save trajectories. Validate tracking quality on several short clips.

Phase 2 — Manual Floor Calibration

Allow the user to select floor reference points. Generate a top-down coordinate system and project tracked people into it.

Phase 3 — Animated Heatmap

Render cumulative and time-window heatmaps and synchronize them with the original video timeline.

Phase 4 — Shelf/Layout Understanding

Detect shelves/racks and convert them into structured 2D geometry. Draw the visible store layout.

Phase 5 — Automatic Map Estimation

Reduce manual calibration by estimating floor plane and scene geometry automatically where visual evidence supports it.

Phase 6 — Analytics + Productization

Add dwell time, zone analytics, exports, job management, authentication, storage, and deployment.

10. MVP Definition

The first demonstrable version should intentionally avoid the hardest problem — fully automatic store reconstruction — and instead prove the end-to-end value chain.

1. Upload a 2–3 minute fixed CCTV clip.
2. Run person detection and multi-object tracking.
3. Let the user mark a small set of floor-plane reference points.
4. Generate a rough top-down map for the visible region.
5. Project customer trajectories onto the map.
6. Render an animated heatmap synchronized with the video.
7. Allow the user to move a timeline slider and inspect the heatmap at any point.
8. Export the heatmap video and underlying JSON data.

This MVP establishes the most important technical foundation. Automatic shelf/layout extraction can then be added without redesigning the customer-tracking and heatmap architecture.

11. Technical Challenges and Mitigation

Challenge	Impact	Mitigation
Occlusion between customers	Tracking ID switches	Use temporal tracking, appearance cues, and tune tracker thresholds
Perspective distortion	Incorrect map positions	Use floor-plane calibration/homography
Only partial store is visible	Incomplete layout	Represent visible boundary and mark unknown regions explicitly
Shelves may be visually similar	Imprecise geometry	Use segmentation/geometric post-processing and confidence scores
Customers stop or overlap	Dwell/trajectory noise	Smooth trajectories and aggregate over time windows
Large videos	Long processing time	Sample frames, batch inference, GPU acceleration, cache intermediate artifacts

12. Privacy and Responsible Use

The system should be designed around anonymous movement analysis rather than identity recognition. For the proposed use case, face recognition and personally identifying information are unnecessary. Raw video should be retained only as long as required by the application's operational requirements, and access should be controlled if the system is deployed with real retail footage.

The output should also communicate that customer trajectories and heatmaps are estimates. Tracking failures, occlusions, and camera limitations can affect the results.

13. Success Criteria

- The system can process a representative 2–3 minute fixed-camera retail clip end-to-end.
- Most visible customers receive stable tracking IDs for useful portions of the video.
- Projected trajectories remain spatially consistent with the visible floor area.
- The 2D map correctly represents the camera-visible region without claiming unseen areas.
- The heatmap changes correctly with time and can be inspected through a timeline.
- The system produces reproducible JSON, image, and video outputs for the same input and configuration.
- The architecture allows shelf/layout detection to be improved independently of customer tracking.

14. Future Extensions

- Multiple CCTV cameras with cross-camera trajectory stitching.
- Automatic camera calibration and floor-plane estimation.
- Automatic entrance/exit detection and customer flow analysis.
- Zone-level dwell-time and visit-frequency analytics.
- Shelf-level traffic analytics.
- Product-facing analysis using shelf segmentation and product detection.
- Store-layout comparison across different days or camera recordings.
- Real-time CCTV processing instead of offline 2–3 minute clips.
- 3D scene reconstruction for environments where a richer spatial model is justified.

15. Final Proposed Architecture

```
Recommended final architecture

Frontend (Angular/React)
↓
Spring Boot API / Job Manager
↓
Python CV Worker
├── OpenCV / FFmpeg — video preprocessing
├── Detection — people + store structures
├── Tracking — persistent customer IDs
├── Geometry — floor-plane / homography mapping
├── Layout Engine — visible shelves + walkable area
└── Heatmap Engine — time-indexed spatial density
↓
Artifacts: store_map.json + tracks.json + heatmap frames + heatmap.mp4
```

Conclusion

The recommended implementation treats the project as a spatial analytics pipeline rather than a single AI model. The most important design decision is to separate what the camera can observe from what the system assumes about the entire store. A calibrated 2D representation, robust person tracking, and time-aware heatmap generation provide a practical foundation. Once those components are reliable, automated shelf and layout reconstruction can be introduced progressively.

The result is a strong computer-vision project with a clear MVP, measurable milestones, and a natural path from a research prototype to a deployable retail analytics application.